

Workflow-Aware Snapshot Restore: Reducing Cold Starts in FaaS Workflows

MSc thesis progress report · Amirkabir University of Technology · September 2026

Title: Improving the function runtime environment in FaaS to reduce workflow cold start

1. The problem

A serverless **workflow** is a chain or graph of functions (for example `resize` → `classify` → `store`). When it has been idle, **every** stage starts cold, one after another, because stage 2 only receives its request when stage 1 finishes. The cold starts **add up along the chain**.

- On our testbed, a 3-stage Spring Boot workflow takes **12.1 s cold vs 35 ms warm**.
- Published measurements show the same linear growth on AWS Step Functions and Azure Durable Functions (Xanadu, Middleware '20; Kulkarni et al. 2025).
- Production traces show this is common: in Azure's workflow traces, 80% of workflows run fewer than 100 times a day, and those see about **50% cold starts** (ORION, OSDI '22). Keep-alive cannot help rare workflows.
- Small containers make it worse. On the JVM, warm-up gets **13-35× slower** from 4 to 0.25 vCPU (our measurement).

Snapshots (CRIU / CRaC, AWS SnapStart) save a started, warmed-up function and restore it much faster than a cold start. But today's snapshot systems work **one function at a time**: each stage is restored only when its request arrives, so a workflow still pays its restores **in series**.

2. The idea

When a cold workflow arrives, restore the snapshots of all its stages in parallel, each timed to be ready just when that stage's input arrives. The workflow's own execution hides the downstream restores.

The orchestrator can do this because it knows the workflow graph and each function's measured timings. It reads **no user data** and does **not predict** when workflows will be called.

```
Restore on demand (today):
resize      [restore][run]
classify                [restore][run]
store                                [restore][run]          -> 3 restores in series

Look-ahead restore (proposed):
resize      [restore][run]
classify    [restore][run]
store       [restore][run]                                -> only 1 restore is visible
```

On the 3-stage Java example (restore 650 ms, run 75 ms): **2.18 s → 0.88 s**.

3. How the approach works

At build time (per function, on every deploy):

1. Measure each function's restore time, run time, memory and image size with a few test runs.
2. Take the snapshot at the right point for the runtime. For Java, also warm it up with the developer's own test requests. Clear random-number state and secrets before the snapshot.
3. Decide **which stages deserve a snapshot**. Some runtimes start fast anyway, and some stages' start-up is already hidden by the workflow. An exact optimisation chooses, under a storage budget or a price.

At run time (each time a cold workflow arrives):

1. Compute, from the graph and the measured times, when each stage will need its sandbox.
2. Start each restore just in time.
3. Keep memory under the platform's budget. Restoring ahead raises the memory *peak*, so a **memory guard** admits restores within the budget. When memory is short, a planner computes the best order.
4. If/else branches: restore a branch ahead only if it is likely enough.

Between runs: keep-alive as usual. When memory is short, evict idle sandboxes by cost-aware caching (GreedyDual-Size-Frequency, from FaasCache/CIDRE). In simulation it beat least-recently-used eviction; evicting whole workflows at once was tested and was worse.

4. Why it is sound (theory, in plain words)

Each result is proved and checked by a verification program (76 automated checks, including "control" checks designed to fail when a claim is false):

- **Timing:** just-in-time look-ahead reaches the **lowest latency any policy can reach**, and uses **no more memory-time** than today's restore-on-demand.
- **Which stages get snapshots:** an exact dynamic program, for a storage budget, a price or a latency target.
- **Memory:** look-ahead raises the peak, but the guard **never exceeds the budget and never deadlocks**. On chains it is never slower than today's behaviour.
- **Uncertain times and branches:** one price ratio decides both how early to restore and whether to restore a branch speculatively.
- **The algorithm is a known problem.** The whole approach is an instance of *resource-constrained project scheduling with time lags*, a classical operations-research problem. Our results are its exactly solvable special cases. The general memory-limited case is NP-hard; a standard branch-and-bound solves typical workflow sizes (≤ 10 stages) in milliseconds.

5. Evidence so far

kind	result
measured (real JVM runs)	cold 12.1 s vs 35 ms warm; deeper snapshots cut the 3-stage workflow to 0.23 s; the vCPU cliff (13–35×)

kind	result
simulated (Azure 2021 trace, 68 workflows, 433 k invocations)	cold workflows: mean 2.34 s → 0.81 s , p99 6.2 s → 1.25 s , at the same memory
simulated (Java chains, cold)	3 / 5 / 8 stages: 2.2 → 0.88 s , 3.7 → 0.99 s , 5.9 → 1.19 s (up to 5× faster)
simulated (memory planner)	under a tight memory budget, planning the restore order exactly: up to 48% faster than a simple memory guard for one workflow, 3–11% for bursts
verified (theory)	76/76 checks pass

6. What is new

- **Snapshot systems** (SnapStart, Prebaking, Pronghorn, REAP, Spice) make *one* restore faster. They do not use the workflow graph, and they combine with our approach.
- **Workflow-aware systems** (Xanadu, ORION) pre-start *cold* containers along the graph. Xanadu already times them just in time. A cold start, however, is usually too long to hide.
- **Ours**: the same timing idea applied to snapshots, which *are* short enough to hide, plus
 - proofs of optimality;
 - which stages deserve a snapshot;
 - handling of the memory budget;
 - the keep-alive consequences.

7. Next steps and risks

1. **Main risk: parallel restores.** The gain assumes several restores can run at once. If they slow each other down (shared disk or CPU), most of the gain disappears. A first test on our CRIU machine measures this (1 week).
2. Real snapshot measurements for one function (restore time, image size, remaining warm-up).
3. Implementation in **Apache OpenWhisk**: an external workflow orchestrator plus a snapshot action image.
4. Evaluation against baselines (restore on demand, cold prewarm, keep-alive, JVM flags) on benchmark workflows (SeBS-Flow) and trace replay.
5. Writing.

8. Assumptions and limits

- The functions' timings are profiled and roughly stable; the theory also covers random timings.
- Restores run in parallel with limited slowdown. This is to be measured, and the theory gives the loss as a function of it.
- The approach helps most where restores are slow (hundreds of ms to seconds, stock CRIU/CRaC) and workflows are deep. With OS-level millisecond restores there is little left to hide.